

Contents

Shopping Agent	1
What this is	1
Submission documents	1
Stack	2
Prerequisites	2
Run it locally	2
1. Clone	2
2. Start Postgres and Redis	2
3. Configure backend	2
4. Configure frontend	3
5. Install dependencies	3
6. Migrate the database	3
7. Start both processes	3
8. Open the app	3
Smoke test	4
Project layout	4
Tracks 2–5	4
Troubleshooting	5
License	5

Shopping Agent

Track 1 submission for the Kasparro Agentic Commerce Hackathon — an AI shopping concierge grounded in the Shopify Global Catalog MCP.

What this is

A buyer types what they want in natural language (“gaming laptop under \$1500 with good battery”). The agent asks one clarifying question only when the missing constraint actually changes the answer, searches the live cross-merchant Shopify catalog, audits products in fast-moving categories for generation freshness, compares 2–4 picks side-by-side, and links straight out to the merchant’s Shop Pay checkout.

Two surfaces:

- **/chat** — the agent loop.
- **/** — a non-AI recommendation feed over the same Shopify Catalog MCP. Browse without spending a free-tier LLM token.

Submission documents

- **PRODUCT.md** — what we built, for whom, why, scope, tradeoffs.
- **TECHNICAL.md** — architecture, implementation, failure handling, limitations.
- **DECISIONS.md** — decision log (“considered X, chose Y, because Z”).
- **CONTRIBUTION.md** — solo build; time split across product vs. engineering.
- **BUILD_PROCESS.md** — running build notebook (source for the four docs above).

- Demo video: add unlisted YouTube / Drive link here.

Stack

- Frontend — Next.js 16 App Router, React, Tailwind, shadcn/ui, Vercel AI SDK 6.
- Backend — Node + tsx, Vercel AI SDK 6, Hono-style route handler in one `server.ts`.
- LLM — OpenRouter (`z-ai/glm-4.5-air:free` primary, free fallbacks) and optional direct DeepSeek (`deepseek-v4-flash`).
- Catalog — Shopify Global Catalog MCP (<https://catalog.shopify.com/api/ucp/mcp>).
- Storage — Postgres 16 + Redis 7 via Docker Compose.
- Web evidence — Playwright + stealth, Bing → DuckDuckGo → Google fallback chain.

Prerequisites

- Node ≥ 20
- pnpm (frontend) — `corepack enable && corepack prepare pnpm@latest --activate`
- npm (backend)
- Docker Desktop (for Postgres + Redis)
- An OpenRouter API key (free tier is fine) — sign up at <https://openrouter.ai/>

Run it locally

1. Clone

```
git clone <this-repo>
cd shopping-agent
```

2. Start Postgres and Redis

```
docker compose up -d
```

Postgres comes up on `localhost:5433`, Redis on `localhost:6379`.

3. Configure backend

```
cp backend/.env.example backend/.env
```

Edit `backend/.env`:

```
PORT=4000
BACKEND_INTERNAL_SECRET=<any random string, e.g. `openssl rand -hex 16`>
POSTGRES_URL=postgres://shopper:shopper@localhost:5433/shopping_agent
REDIS_URL=redis://localhost:6379
OPENROUTER_API_KEY=<your-openrouter-key>
OPENROUTER_REFERRER=http://localhost:3000
# optional
DEEPSEEK_API_KEY=
SHOPIFY_CATALOG_MCP_URL=https://catalog.shopify.com/api/ucp/mcp
SHOPIFY_UCP_AGENT_PROFILE=https://shopify.dev/ucp/agent-profiles/examples/2026-04-08/va
```

```
# optional: residential proxy for Google search on datacenter IPs
SEARCH_PROXY_URL=
```

4. Configure frontend

```
cp frontend/.env.example frontend/.env.local
```

Edit frontend/.env.local:

```
AUTH_SECRET=<generate with `openssl rand -base64 32`>
POSTGRES_URL=postgres://shopper:shopper@localhost:5433/shopping_agent
REDIS_URL=redis://localhost:6379
OPENROUTER_API_KEY=<your-openrouter-key>
OPENROUTER_REFERRER=http://localhost:3000
SHOPIFY_CATALOG_MCP_URL=https://catalog.shopify.com/api/ucp/mcp
SHOPIFY_UCP_AGENT_PROFILE=https://shopify.dev/ucp/agent-profiles/examples/2026-04-08/va
BACKEND_URL=http://localhost:4000
BACKEND_INTERNAL_SECRET=<same value you set in backend/.env>
# AI_GATEWAY_API_KEY only required if you are NOT on Vercel
AI_GATEWAY_API_KEY=
# Vercel Blob token only required if you want attachment uploads in dev
BLOB_READ_WRITE_TOKEN=
```

5. Install dependencies

```
# backend
```

```
cd backend && npm install
```

```
# frontend
```

```
cd ../frontend && pnpm install
```

6. Migrate the database

From the frontend directory (the Drizzle config lives there):

```
pnpm drizzle-kit migrate
```

7. Start both processes

In two terminals, from repo root:

```
# terminal 1 - backend
```

```
cd backend && npm run dev
```

```
# terminal 2 - frontend
```

```
cd frontend && pnpm dev
```

8. Open the app

- Chat: <http://localhost:3000/chat>

- Recommendation feed: <http://localhost:3000/>

Click Continue as guest on the auth screens — no signup needed for the demo path.

Smoke test

A quick sanity flow:

1. Open <http://localhost:3000/chat>.
2. Send: “gaming laptop under \$1500 with good battery”
3. Agent should ask one clarifying question (use case) as quick-reply chips.
4. Pick a chip — agent searches Shopify Catalog MCP and renders 2–6 product cards.
5. Send: “compare the top 3” — should render a native comparison table.
6. Click Buy on the cheapest in-stock card — should open the merchant’s Shop Pay / product page in a new tab.

Project layout

```
shopping-agent/
├── frontend/
│   ├── app/(chat)/
│   ├── app/page.tsx
│   ├── components/chat/
│   │   └── shopping/
│   ├── hooks/use-active-chat.tsx
│   ├── lib/db/
│   ├── lib/feed/
│   └── lib/ai/models.ts
├── backend/
│   ├── src/server.ts
│   └── src/lib/
│       ├── agents/registry.ts
│       ├── agents/track1-shopping/
│       │   ├── shopify/catalog.ts
│       │   ├── ai/providers.ts
│       │   └── cache/redis.ts
├── docker-compose.yml
├── product_extractor/
└── not part of the runtime
    ├── PRODUCT.md
    ├── TECHNICAL.md
    ├── DECISIONS.md
    ├── CONTRIBUTION.md
    └── BUILD_PROCESS.md
```

Next.js 16 App Router app
/chat shell + chat by id
Recommendation feed homepage
Chat surface
Generative UI parts
useChat wrapper, transport, persistence sync
Drizzle schema + queries (Postgres)
Typed client around backend /api/feed/*
Model picker UI options
Node + tsx server
All routes – chat, messages, feed, votes
Track 1 wired, Tracks 2–5 stubs
System prompt + 12 tools + freshness lib
Shopify Catalog MCP JSON-RPC client
OpenRouter + direct DeepSeek
getOrSet helper
Postgres 16 + Redis 7
Legacy CSV pipeline –

Tracks 2–5

backend/src/lib/agents/registry.ts already maps track2-checkout-recovery, track3-checkout-copilot, track4-support, track5-rep-optimizer to stubs that share

the Track 1 AgentDefinition shape. They can be filled in with their own system prompts + tools without touching the routing or persistence layer.

Troubleshooting

- **ECONNREFUSED 127.0.0.1:5433** — Postgres container is not up. `docker compose ps` and `docker compose up -d`.
- **ECONNREFUSED 127.0.0.1:6379** — same for Redis.
- **Shopify Catalog MCP search_catalog failed (4xx)** — check that your machine has outbound HTTPS to `catalog.shopify.com`. No auth token is required for the public global endpoint.
- **OpenRouter 429 rate limit** — switch to a different model in the chat header (model picker), or wait the cooldown.
- **AUTH_SECRET** missing — frontend will refuse to start. Generate with `openssl rand -base64 32`.
- Google CAPTCHA in **webSearch** — set `SEARCH_PROXY_URL` to a residential proxy URL in `backend/.env`. DuckDuckGo / Bing legs of the chain usually succeed without it.

License

See `frontend/LICENSE`.